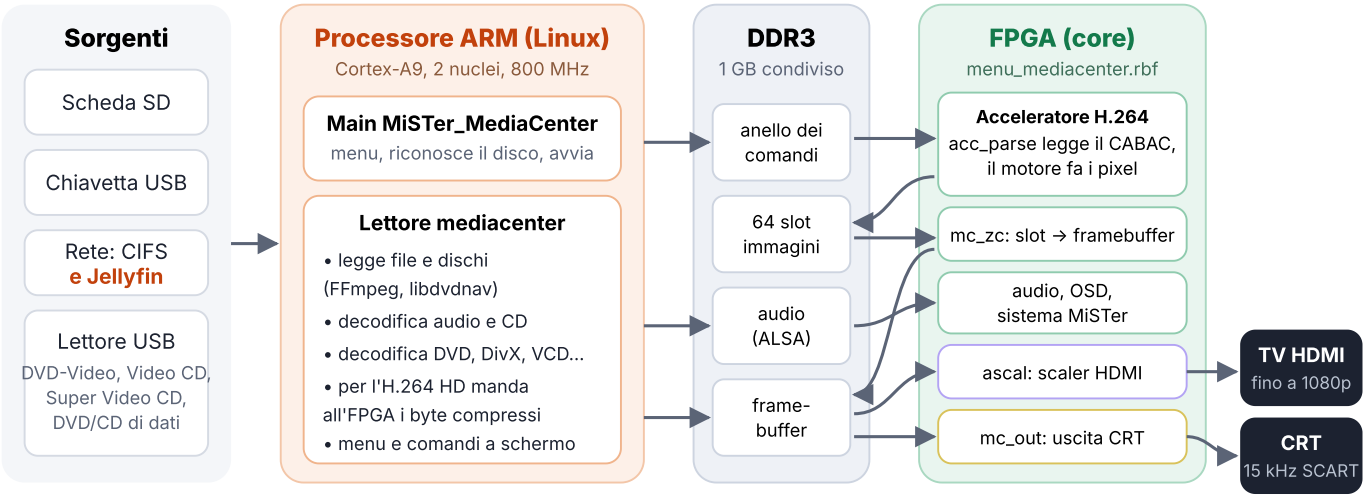
DOCUMENTAZIONE COMPLETA

# Come funziona MiSTer MediaCenter 4.5, dal disco allo schermo

Il chip FPGA, il lettore sul processore ARM e come si passano il lavoro, il viaggio di ogni immagine verso HDMI e CRT, le novità della 4.5 e la ricetta per ricostruire tutto da zero.




## Il sistema in una figura




ARM e FPGA non si parlano mai direttamente: si lasciano dati e messaggi in zone fisse della memoria DDR3.

## Carta d'identità della versione 4.5

| | |
|---|---|
| Release | **v4.5**, 12 novembre 2026, commit `307a7c5` |
| Lettore ARM | `mediacenter` build **224** |
| Core FPGA | `menu_mediacenter.rbf`, seme **12**<br>MD5 `7cb8175f995d5b72a49c2ef032fbc963`<br>lo stesso file della 4.0 |
| Scheda | Terasic DE10-Nano, chip Intel Cyclone V SE `5CSEBA6U23I7` |
| Novità | Jellyfin, schermata audio, CD audio, Opus e moduli tracker, Main dell'11/11, nuovo nome dello script |

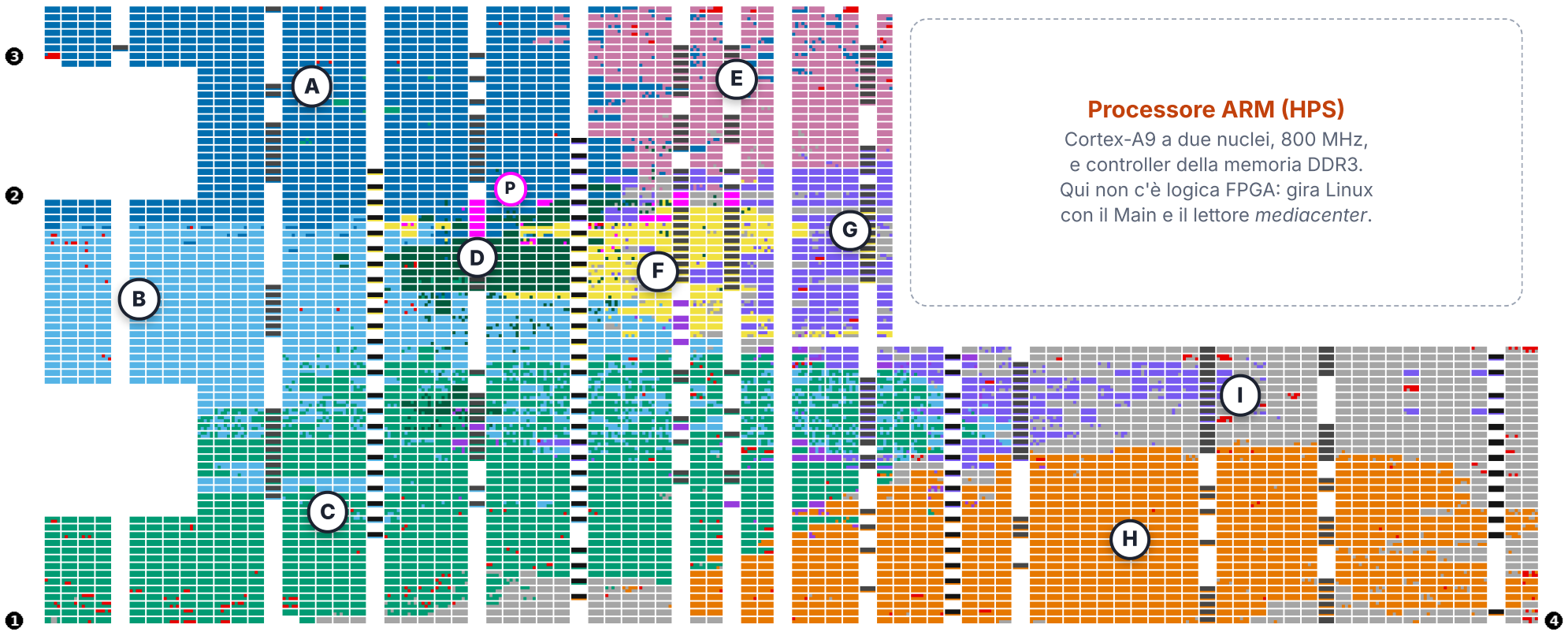
## Le pagine


| | |
|---|---|
| 2 | La mappa del chip (pagina orizzontale) |
| 3 | Le novità della 4.5 e i legami ARM–FPGA |
| 4–5 | Risorse del chip, clock e blocchi |
| 6 | Il viaggio 1: dalla sorgente al lettore |
| 7 | Il viaggio 2: chi decodifica che cosa |
| 8 | Il viaggio 3: la strada dell'ARM e l'audio |
| 9 | Il viaggio 4: la strada dell'FPGA |
| 10 | Memoria condivisa e messaggi |
| 11 | Le uscite HDMI e CRT, i registri |
| 12–13 | Il viaggio 5: Jellyfin |
| 14 | La schermata audio e l'equalizzatore |
| 15 | CD audio, formati, Main, script |
| 16–17 | Ricostruire tutto da zero |
| 18–19 | Glossario, mappa dei sorgenti |


**Come leggere gli schemi di questo documento.** Gli stessi colori valgono in tutte le pagine: arancio chiaro = processore ARM (programmi), grigio-blu = memoria DDR3 condivisa, verde = FPGA (circuiti). Le pastiglie 108 100 85,7 27 indicano il clock di un blocco in MHz. Le frecce seguono i dati. Ogni pagina indica in fondo i file del repository da cui vengono le informazioni.

# 2 Il chip FPGA visto dall'alto: core seme 12 (4.0 e 4.5)

Disegnata dalla posizione reale delle 120.318 celle nel database di Quartus 17.0.2




| | | | |
|---|---|---|---|
| **A** | Filtro anti-blocchi (acc_dbk) 100 | 6.628 ALM · 15,8% | |
| **B** | Compensazione del moto 100 | 6.094 · 14,6% | |
| **C** | Motore: IDCT, intra, ricostruz. 100 | 8.886 · 21,2% | |
| **D** | Coda e deviatore A6 (mc_acc) 100 | 1.033 · 2,5% | |
| **E** | Copia nel framebuffer (mc_zc) 108 | 2.506 · 6,0% | |
| **F** | Uscita CRT 15 kHz (mc_out) 27 | 1.022 · 2,4% | |
| **G** | Scaler HDMI (ascal) HDMI | 2.270 · 5,4% | |
| **H** | acc_parse: lettore CABAC 85,7 | 7.278 · 17,4% | |
| **I** | Sistema MiSTer, resto core | 5.594 · 13,4% | |
| **P** | Ponti 108 ↔ 100 MHz · ponti 100 ↔ 85,7 | 48 + 166 | |
| | ALM completamente vuoti | 345 · 0,8% | |
| | Righe scure: memorie M10K · nere: DSP | | |

### Come si legge

Ogni quadratino è un **ALM**, il mattoncino di logica dell'FPGA: fa un piccolo calcolo e ricorda qualche bit. Il chip ne ha 41.910, in gruppi di 10 (i rettangoli 5×2, detti LAB). Il colore dice a quale parte del core appartiene l'ALM. Le pastiglie colorate sono i clock: 100 = 100 MHz e così via.

### I generatori di clock (numeri neri ai bordi)

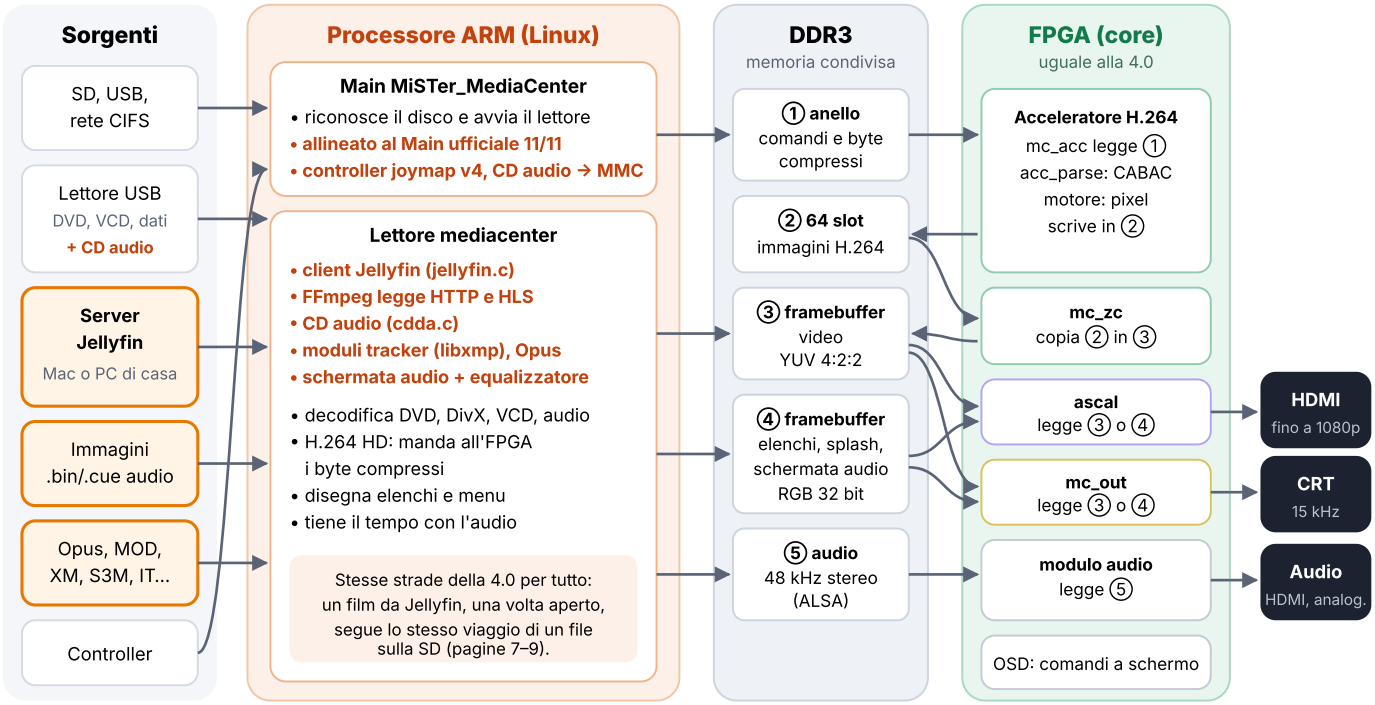
❶ PLL HDMI, riprogrammato a ogni cambio di risoluzione. ❷ PLL dell'acceleratore: 100 MHz al motore, 85,7 MHz ad acc_parse. È fissato lontano dal ❶: quando erano vicini l'HDMI andava fuori range a 720p. ❸ PLL audio, 24,576 MHz. ❹ PLL del core: 108 e 27 MHz.

### Cosa ci dice

Il chip è pieno: restano solo 345 ALM vuoti, sparsi. I numeri della legenda contano ogni ALM toccato anche in parte (41.525). Quartus conta 39.305 ALM necessari (94%): la differenza sono ALM usati solo a metà. Le parti che si parlano molto stanno vicine: il motore a sinistra, le uscite video al centro vicino all'ARM, acc_parse in un blocco compatto in basso a destra.

# 3 Le novità della 4.5 e come si legano ARM e FPGA

La 4.5 cambia solo i programmi che girano sul processore ARM: il lettore `mediacenter` e il Main. Il core FPGA è lo stesso file della 4.0, byte per byte. Le novità usano le strade che l'FPGA già offriva.




**Legenda.** In arancio scuro le cose nuove della 4.5; le caselle con il bordo arancio sono sorgenti nuove. I numeri ①–⑤ sono le zone della memoria DDR3 attraverso cui ARM e FPGA si passano il lavoro: non si parlano mai direttamente.

| Novità | Dove gira | Cosa usa dell'FPGA | Pagina |
|---|---|---|---|
| **Client Jellyfin** | lettore, `jellyfin.c` | acceleratore H.264 (file originali e conversioni del server), uscite HDMI e CRT | 12–13 |
| **Schermata audio con equalizzatore** | lettore, `audio_ui.c` | framebuffer RGB ④, ascal e mc_out, modulo audio | 14 |
| **CD audio dentro MMC** | Main (riconosce) + lettore, `cdda.c` | modulo audio, schermata audio | 15 |
| **Opus e moduli tracker** | lettore: FFmpeg e libxmp, `modplay.c` | modulo audio, schermata audio | 15 |
| **Main allineato all'ufficiale** | Main | niente di nuovo: carica lo stesso core | 15 |
| **Script e immagine nuovi** | script, lettore ( `splash.h` ) | framebuffer ④ | 15 |

## Le quattro strade fra ARM e FPGA, in breve

**1 · Video decodificato dall'ARM** (DVD, DivX, Video CD...): l'ARM fa i pixel e li copia in ③; l'FPGA converte i colori e adatta la misura per HDMI e CRT.

**2 · Video H.264 HD con l'acceleratore** (anche da Jellyfin): l'ARM scrive in ① i byte compressi; l'FPGA ricostruisce le immagini in ②, `mc_zc` le copia in ③ all'ora giusta.

**3 · Schermate disegnate** (elenchi, menu di Jellyfin, splash, schermata audio): l'ARM disegna in ④ a 32 bit; l'FPGA le mostra; i comandi di riproduzione li mescola il core (OSD).

**4 · Audio**: l'ARM decodifica, con la musica applica l'equalizzatore, e manda i campioni ad ALSA (⑤); il modulo audio del core li porta a HDMI, uscita analogica e S/PDIF.

# 4 Quanto spazio usa ogni parte

L'FPGA ha quattro tipi di risorse: la logica è quasi finita, memorie e moltiplicatori hanno ancora margine. Il core della 4.5 è lo stesso della 4.0, quindi questi numeri valgono per tutte e due.

Logica (ALM)
**94%**
39.305 su 41.910

Memorie M10K
**43%**
238 su 553 blocchi da 10 kbit

Moltiplicatori DSP
**74%**
83 su 112

Piedini
**46%**
145 su 314

**ALM**: il mattoncino di logica, fa piccoli calcoli e ricorda 2–4 bit. **Registri**: i bit di memoria dentro gli ALM. **M10K**: memorie interne da 10 kbit, per tabelle e righe di immagine. **DSP**: moltiplicatori veloci, per filtri e calcoli sui pixel. Valori ufficiali di Quartus (report *Fitter Resource Utilization by Entity*); un modulo comprende i suoi sotto-moduli. Sommando i moduli, che usano anche frazioni di ALM, il totale arrotondato è 39.306.

| Parte del core (file) | Clock (MHz) | ALM | Registri | M10K | DSP |
|---|---|---|---|---|---|
| **Tutto il chip (sys_top)** | | **39.306** | **54.398** | **238** | **83** |
| **Core MediaCenter** (menu.sv) | | 32.804 | 44.111 | 175 | 53 |
| **Acceleratore H.264** (mc_acc.sv e sotto-moduli) | 100 | 28.818 | 38.964 | 118 | 33 |
| Coda dei comandi, deviatore A6, tabelle | 100 | 951 | 1.823 | 7 | |
| Ponti 108 ↔ 100 MHz (mc_afifo) | 108 100 | 44 | 92 | 9 | |
| **Motore** (acc_engine.sv): intra, comandi, scrittura | 100 | 5.005 | 7.701 | 18 | 3 |
| Filtro anti-blocchi (acc_dbk.sv) | 100 | 6.417 | 11.474 | 26 | |
| Compensazione del moto (acc_mc.sv) | 100 | 6.339 | 6.848 | 11 | 21 |
| Trasformata inversa (acc_idct.sv) | 100 | 1.627 | 3.052 | | |
| Forza del filtro (acc_bs.sv) | 100 | 913 | 582 | 8 | |
| Residui e ricostruzione (acc_resid, acc_recon) | 100 | 448 | 273 | 12 | |
| **Lettore CABAC** acc_parse + cab_dec | 85,7 | 6.872 | 6.837 | 28 | 9 |
| 4 ponti 100 ↔ 85,7 MHz (mc_afifo) | 100 85,7 | 103 | 227 | 7 | |
| Copia nel framebuffer (mc_zc.sv) | 108 | 2.299 | 2.942 | 29 | |
| Uscita CRT (mc_out.sv) | 27 108 | 970 | 1.066 | 28 | 20 |
| Collegamento con l'ARM (hps_io), bus, SDRAM | 108 | 705 | 1.139 | | |
| **Framework MiSTer** (cartella sys/) | | 6.501 | 10.287 | 63 | 30 |
| Scaler HDMI (ascal.vhd) | HDMI 108 | 2.034 | 3.775 | 42 | 22 |
| Audio (audio_out, alsa) | 24,6 | 1.189 | 1.601 | | 8 |
| Menu a schermo OSD (osd.v ×2) | video | 1.008 | 1.298 | 16 | |
| Controllo PLL HDMI, memoria, uscite, resto | vari | 2.270 | 3.613 | 5 | |

**ALM per parte (su 39.306)**



- Motore (con filtro e moto) 20.800 · 53%
- Lettore CABAC acc_parse 6.872 · 17%
- Copia nel framebuffer 2.299 · 6%
- Scaler HDMI 2.034 · 5% · Uscita CRT 970 · 2%
- Framework, ARM, bus, audio, OSD 5.185 · 13%
- Coda e ponti dell'acceleratore 1.146 · 3%

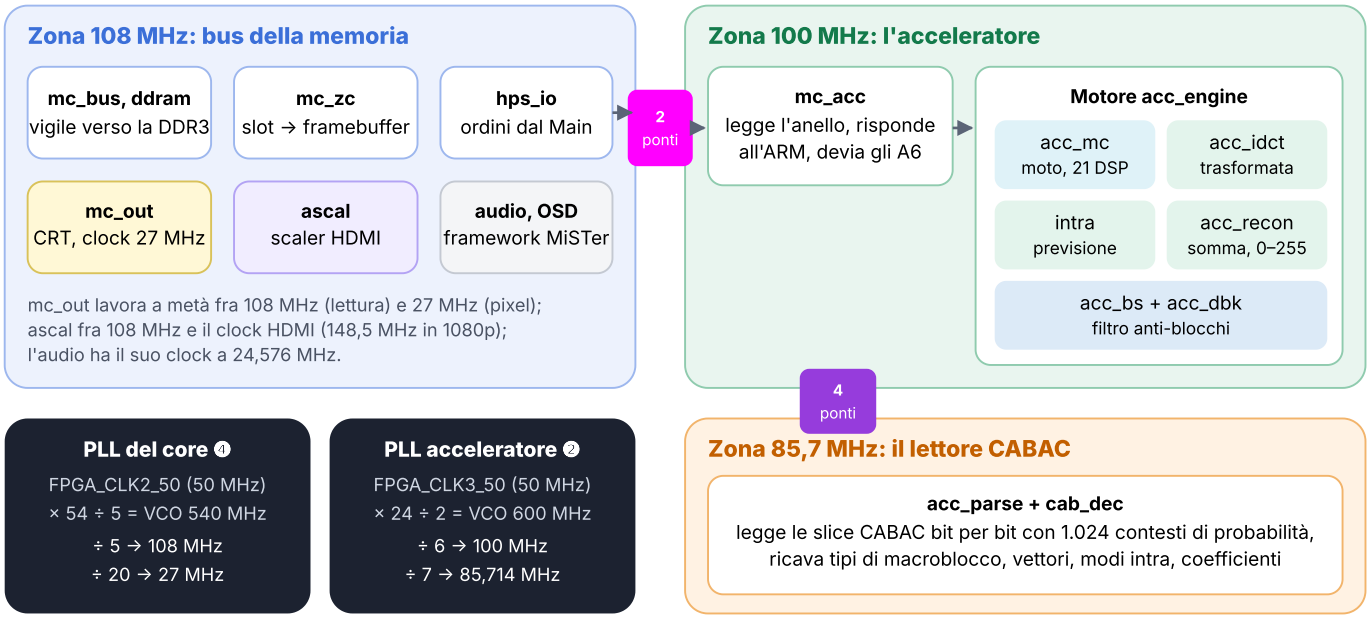
**Cosa vuol dire.** Motore e lettore CABAC insieme occupano il 70% della logica: sono il cuore delle versioni 4.0 e 4.5. Siccome restano pochi ALM liberi, ogni funzione nuova nell'FPGA va disegnata per risparmiare logica, oppure togliendo qualcos'altro. Memorie e moltiplicatori invece hanno ancora spazio.

**Altri numeri:** 4.187 LAB usati su 4.191; 56.718 funzioni logiche (ALUT); 1.399.503 bit di memoria su 5.662.720.

Fonte: report del fitter di Quartus 17.0.2, seme 12

# 5 I clock e i blocchi del core

Un circuito digitale avanza a colpi di clock, come un'orchestra che segue il direttore. Nel core (lo stesso nella 4.0 e nella 4.5) ci sono zone che battono a ritmi diversi; i dati passano da una zona all'altra solo attraverso piccole code chiamate ponti.




## Perché tre zone

Con il chip pieno al 94% i fili sono lunghi e un segnale impiega più tempo ad arrivare. Il motore non chiudeva i tempi né a 108 né a 104 MHz, a 100 MHz sì; acc_parse ha catene di calcolo lunghe (ogni bit dipende dal precedente) e ha un suo clock più lento, 85,7 MHz. Il bus della memoria e le uscite video restano a 108 MHz come nella 3.6.

I **ponti** (`mc_afifo`) sono code con i contatori in codice Gray: fra un numero e il successivo cambia un solo bit, così chi legge da un altro clock non vede mai un valore a metà. Due ponti collegano bus e acceleratore, quattro collegano motore e acc_parse.

## Gli altri due generatori

**PLL HDMI ❶** (da FPGA_CLK1_50): il Main lo riprogramma a ogni risoluzione; Quartus controlla i tempi sul caso più veloce, 148,5 MHz del 1080p. **PLL audio ❸** (da FPGA_CLK1_50, PLL frazionario, VCO 417,792 MHz ÷ 17): 24,576 MHz per I2S, S/PDIF e uscita analogica.

**Posizione fissa.** Il PLL dell'acceleratore è bloccato in `FRACTIONALPLL_X0_Y56` nel file `menu.qsf` e prende i 50 MHz dal piedino FPGA_CLK3_50. Messo accanto al PLL HDMI, disturbava la sua riprogrammazione e l'HDMI andava fuori range a 720p.

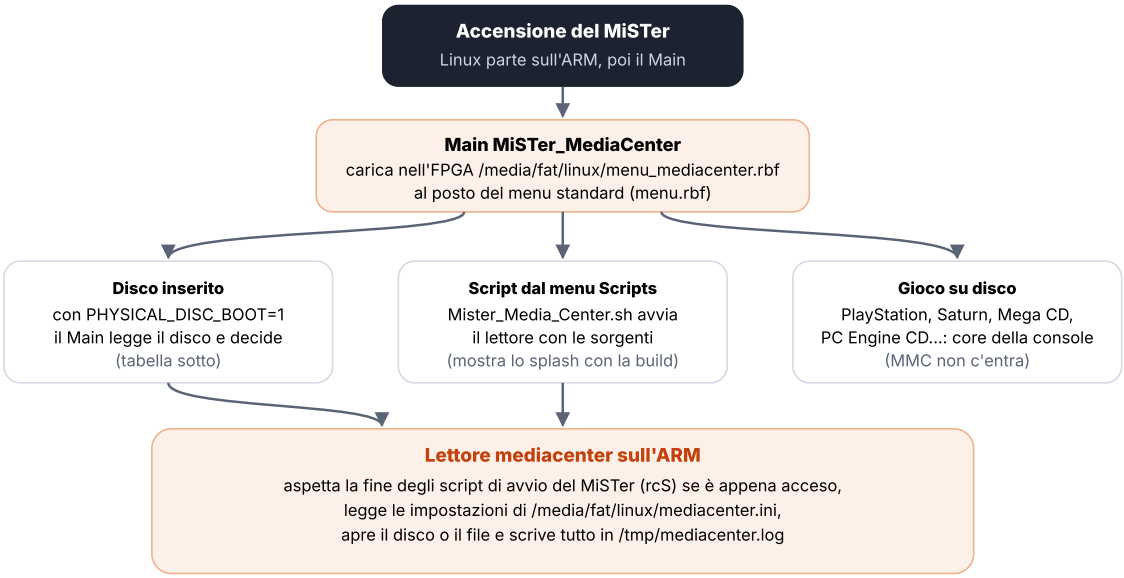
### I tempi del seme 12 (margine peggiore in nanosecondi, positivo = chiude)

| Clock | Periodo | Lento 100 °C | Lento −40 °C | Veloce 100 °C | Veloce −40 °C | Hold peggiore |
|---|---|---|---|---|---|---|
| Bus 108 MHz | 9,259 | +0,273 | +0,220 | +4,128 | +4,979 | +0,075 |
| Motore 100 MHz | 10,000 | +0,318 | +0,319 | +4,548 | +5,440 | +0,025 |
| acc_parse 85,7 MHz | 11,667 | +0,347 | +0,608 | +5,477 | +6,442 | +0,122 |
| Video CRT 27 MHz | 37,037 | +0,577 | +0,707 | +4,612 | +5,284 | +0,104 |
| HDMI 148,5 MHz | 6,734 | +0,014 | +0,221 | +2,602 | +3,296 | +0,120 |
| Audio 24,576 MHz | 40,690 | +14,430 | +13,835 | +27,432 | +29,077 | +0,125 |

Quartus controlla ogni filo nel chip più lento e in quello più veloce che Intel produce, a 100 °C e a −40 °C. Setup: il segnale arriva prima del colpo di clock successivo. Hold: non cambia troppo presto dopo il colpo. Sul codice finale sono stati provati 9 semi: solo il 12 chiude in tutte e quattro le condizioni.

# 6 Il viaggio 1: dalla sorgente al lettore

Prima che si veda un'immagine, il MiSTer deve caricare il core giusto, capire che cosa c'è nel lettore o quale file è stato scelto, e avviare il programma `mediacenter` sull'ARM.




## Che cosa decide il Main per un disco

| Nel lettore | Come lo riconosce | Chi lo apre |
|---|---|---|
| DVD-Video | c'è `VIDEO_TS/VIDEO_TS.IFO` | mediacenter /dev/sr0 |
| Video CD, Super Video CD | settore 150 inizia con VIDEO_CD, SUPERVCD o HQ-VCD | mediacenter |
| DVD di dati con video o musica | file multimediali sul disco | mediacenter (elenco file) |
| CD audio | solo tracce audio | **mediacenter** (nuovo), o MiSTer Hi-Fi (pagina 15) |
| Giochi | firma della console | core della console |

## Le sorgenti nell'elenco del lettore

| Voce | Cartella |
|---|---|
| Disco nel lettore | il primo `/dev/srN` |
| SD: Video, SD: Musica | `/media/fat/Video`, `/media/fat/Music` |
| SD: tutta la scheda | `/media/fat` |
| USB 0...7 | `/media/usbN`, senza doppioni |
| Rete (CIFS) | `/media/fat/cifs` |
| Jellyfin | server di casa, pagine 12–13 |
| Regola CRT | dimensioni e posizione sul televisore a tubo |

Novità della 4.5: Jellyfin arriva dalla rete (pagine 12–13); CD audio e moduli tracker passano da un file WAV costruito al volo (pagina 15).

## Come vengono letti i dati

**File** (MP4, MKV, AVI, MPEG, Ogg, ASF, MP3, FLAC...)
FFmpeg apre il contenitore e lo divide in pacchetti video, audio e sottotitoli, ognuno con il suo orario.

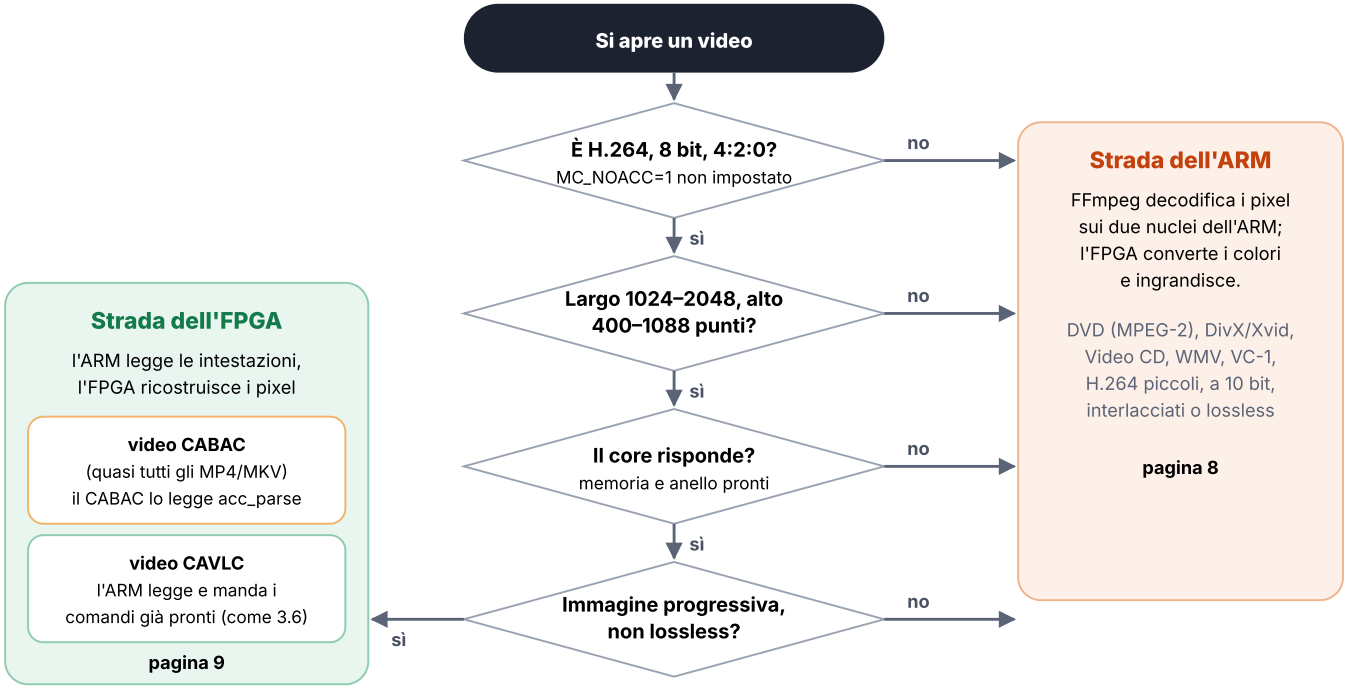
**DVD-Video**
libdvdnav segue menu, titoli e capitoli e passa blocchi da 2048 byte; `dvd.c` li divide in video MPEG-2, audio e sottotitoli. I dischi protetti usano libdvdcss da `/media/fat/dvdcss/`.

**Video CD e Super Video CD**
Il lettore legge i settori delle tracce video direttamente dal disco e passa il flusso MPEG-1 o MPEG-2 a FFmpeg, come per i file.

Per i DVD il lettore usa un thread di lettura su un nucleo, così una lettura lenta dal disco non ferma l'immagine. Tutti i decoder sono di FFmpeg 6.1.2, compilato solo con i formati che servono.

# 7 Il viaggio 2: chi decodifica che cosa

Per ogni video il lettore sceglie una strada. L'ARM sa decodificare tutto, ma è lento con l'H.264 in alta definizione; l'FPGA è velocissimo ma sa fare solo l'H.264 progressivo a 8 bit. La scelta si fa all'apertura del video, anche per i video da Jellyfin.




L'ultima domanda la fa FFmpeg modificato, immagine per immagine: se arriva un'immagine interlacciata, lossless, a più di 8 bit o 4:2:2, l'acceleratore si spegne e il lettore riapre il video sull'ARM dallo stesso punto. Il video lossless di prova (intro Sega) passa così all'ARM e si vede correttamente.

## CABAC sull'FPGA: quando sì e quando no

| Situazione | Chi legge il CABAC |
|---|---|
| Core 4.0 (bit 22 della parola 20 a 1) | **acc_parse nell'FPGA** |
| `MC_NOCABAC=1` in mediacenter.ini | l'ARM, come nella 3.6 |
| Core 3.6 o più vecchio | l'ARM, come nella 3.6 |
| acc_parse trova una slice rovinata | l'ARM, fino alla prossima IDR |
| Video CAVLC | non c'è CABAC: l'ARM legge i dati |

## Che cosa vuol dire, in numeri

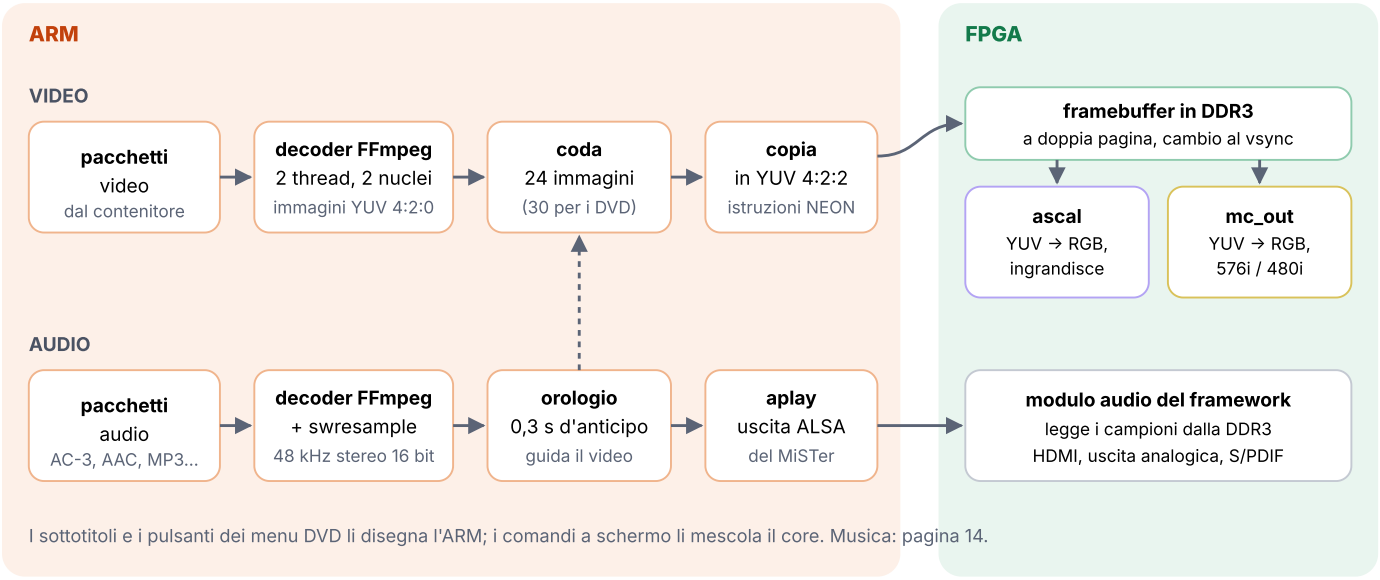
| Video (H.264) | ARM 3.6 | ARM 4.0 |
|---|---|---|
| HTC, 1920×1080, 30 fps | 26,7 ms | **0,4 ms** |
| One Piece 940, 1920×1080 | 12,3 ms | **0,3 ms** |
| Inuyasha, 1440×810 | 5,6 ms | **0,4 ms** |
| Tears of Steel, 1728×720 | 3,8 ms | **0,3 ms** |
| One Piece 938, 1280×720 | 1,7 ms | **0,3 ms** |

Lavoro medio dell'ARM per immagine, misurato sul MiSTer il 9 novembre 2026. Il tempo disponibile è 41,7 ms a 24 fps e 33,4 ms a 30 fps.

**Perché non mettere tutto nell'FPGA?** Il chip è già pieno al 94%: un decoder MPEG-2 o DivX non ci starebbe accanto all'H.264, e non servirebbe. Un DVD (720×576) o un DivX sono leggeri: l'ARM li decodifica senza fatica (nelle prove del 9/11: DivX con 1 immagine in ritardo in 54 s, DVD originale con 4 in ritardo su 4.247). L'FPGA serve dove l'ARM non ce la fa, cioè l'H.264 in alta definizione, che ha fino a 8.160 macroblocchi per immagine contro i 1.620 di un DVD.

# 8 Il viaggio 3: la strada dell'ARM e l'audio

Per DVD, DivX, Video CD e gli altri formati l'ARM fa la parte pesante, la decodifica; all'FPGA restano la conversione dei colori e l'ingrandimento verso il televisore. L'audio segue sempre questa strada, per tutti i video.




I sottotitoli e i pulsanti dei menu DVD li disegna l'ARM; i comandi a schermo li mescola il core. Musica: pagina 14.

## I passi, in parole semplici

1. **Decodifica.** FFmpeg trasforma ogni pacchetto in un'immagine YUV (luminosità e due componenti di colore). Usa due thread, uno per nucleo dell'ARM.
2. **Coda.** Le immagini aspettano il loro turno in una coda di 24 (30 per i DVD): è la riserva per le scene pesanti. All'avvio il lettore aspetta di averla piena, al massimo 2 secondi.
3. **Copia.** Quando è l'ora, l'ARM copia l'immagine nel framebuffer in formato YUV 4:2:2, metà dei byte dell'RGB. Se l'immagine è più larga dello schermo la restringe durante la copia con le istruzioni NEON.
4. **Colori e misura.** Il core converte YUV in RGB: lo scaler `ascal` per l'HDMI, `mc_out` per il CRT. Lo scaler sa solo ingrandire, mai rimpicciolire.

## Chi tiene il tempo

L'audio è l'orologio. Il lettore manda i campioni ad `aplay` con circa 0,3 s d'anticipo e sa sempre quale istante si sta ascoltando. Ogni immagine si mostra quando arriva il suo orario: se è in ritardo il lettore la salta, e nel log compare come *non mostrata*.

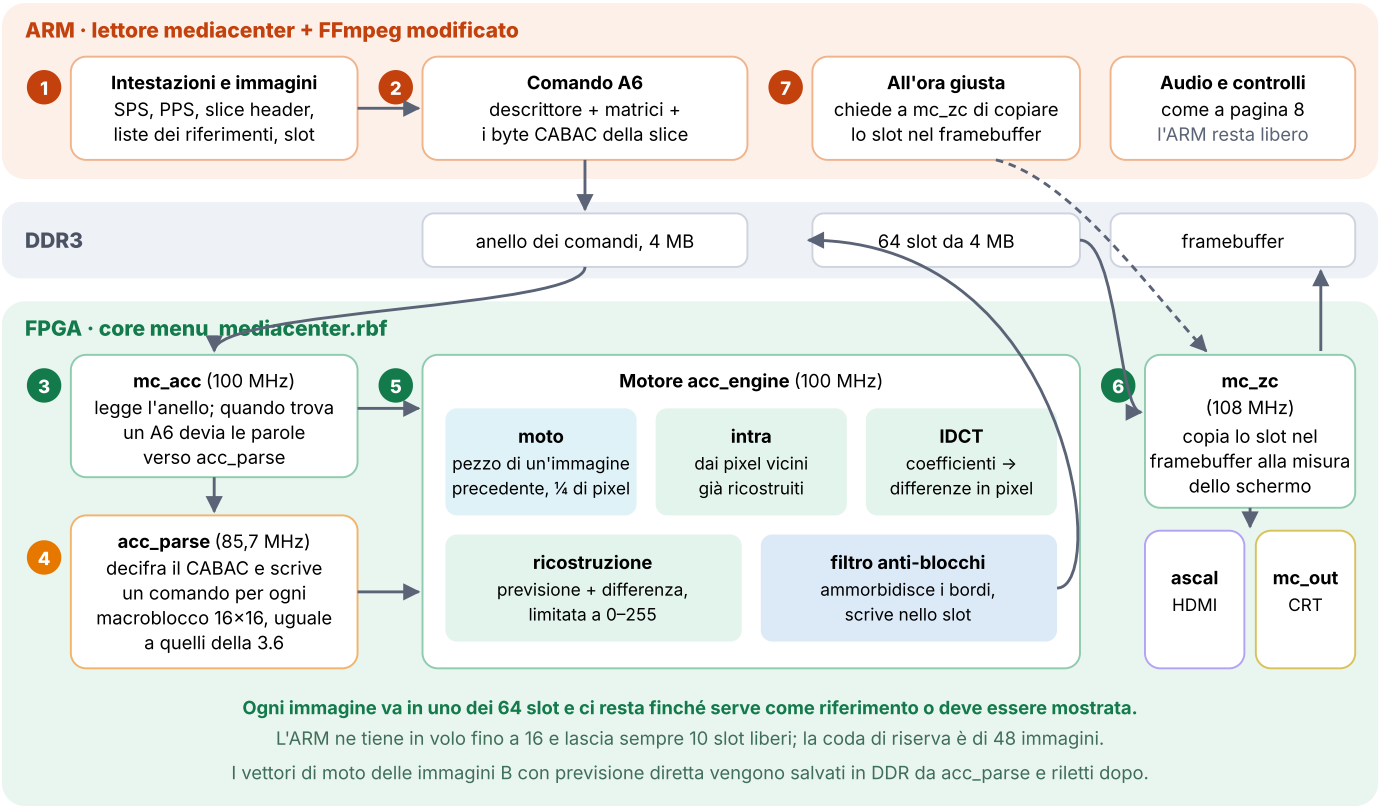
## Il caso Video CD e DVD

Video CD (MPEG-1 352×288) e DVD (MPEG-2 720×576) sono piccoli e l'ARM li decodifica senza fatica. Per i DVD il lettore usa un framebuffer della stessa misura del video, così sul CRT ogni riga del DVD diventa una riga del televisore.

> Il core fa anche da **zero-copy** per l'ARM: con un 1080p decodificato dall'ARM, il blocco `mc_zc` può copiare lui l'immagine nel framebuffer. `MC_NOZC=1` lo spegne.

# 9 Il viaggio 4: la strada dell'FPGA

Per un video H.264 in alta definizione l'ARM fa solo da postino: legge le intestazioni, prepara un piccolo descrittore e passa all'FPGA i byte compressi. Tutto il resto, fino ai pixel sullo schermo, lo fa il core.




## Perché è così veloce

Il CABAC è il modo più compresso in cui l'H.264 scrive i dati: ogni bit dipende da quelli prima, quindi va letto uno dopo l'altro. Per un processore è il lavoro più pesante della decodifica. Nell'FPGA lo fa `cab_dec`, un circuito dedicato con le tabelle di probabilità al suo interno, mentre l'ARM resta libero.

Prima l'ARM scriveva circa 1 MB di comandi per ogni immagine 1080p. Ora scrive solo i byte compressi, 10–20 KB: anche la memoria DDR3 è meno affollata e l'attesa del motore scende (One Piece 1080p da 14,7 a 9,0 ms per immagine).

## Numeri del motore

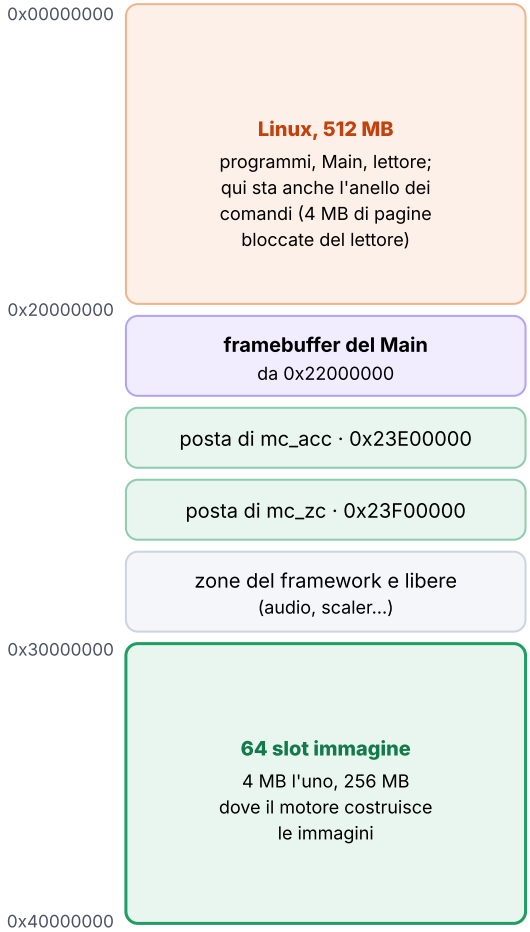
| | |
|---|---|
| Formati | H.264 progressivo 4:2:0 8 bit, profili Main e High (trasformata 8×8, matrici) |
| Immagine massima | 8.192 macroblocchi (1080p ne ha 8.160) |
| Slot | 64 da 4 MB da `0x30000000`, righe Y da 2048 byte |
| Ridimensionamento | 1080p → 1280 o 960 punti; 810p → 960; altre misure ricampionate |
| Errori | 0 immagini perse dall'FPGA e 0 errori CABAC in tutte le prove del 9/11 |

## Un'immagine 1080p in numeri (One Piece 940, 24 fps, prova del 9/11)

| Passo | 3.6 | 4.0 | Che cosa vuol dire |
|---|---|---|---|
| Lavoro dell'ARM per immagine | 12,3 ms | **0,3 ms** | il processore è quasi fermo: resta tempo per audio e menu |
| Dati scritti per l'FPGA | 1.259 KB | **15 KB** | circa 85 volte meno traffico verso la memoria |
| Ritmo dell'FPGA | 40,3 ms | 40,5 ms | un'immagine ogni 41,7 ms circa, come chiede il video |
| Tempo del motore senza comandi | 13,1 ms | 22,3 ms | il motore finisce prima e aspetta: c'è margine |
| Attesa della memoria DDR3 | 14,7 ms | **9,0 ms** | meno code sulla memoria condivisa |

# 10 La memoria condivisa e i messaggi

Il DE10-Nano ha 1 GB di DDR3. Linux usa la prima metà; la seconda è dell'FPGA. ARM e core si scambiano tutto attraverso zone a indirizzo fisso, come cassette della posta.




**Come si legge.** Gli indirizzi sono fisici, uguali per ARM e FPGA. Il lettore li raggiunge con `/dev/mem`; il core con il suo accesso diretto alla DDR3 (porte f2sdram). L'anello sta nella memoria del lettore, che Linux sparpaglia in pagine da 4 KB: una tabella delle pagine dice al core dove si trova ognuna.

**La posta di mc_zc** (firma `MCZC`) riceve gli ordini di copia: quale slot, quale pagina del framebuffer, a che misura. Da +0xC0000 contiene le tabelle delle pagine dei 64 slot.

## L'anello dei comandi

Una coda circolare di 4 MB, come una pista: il lettore scrive in fondo e sposta il suo puntatore; mc_acc legge dall'inizio e dice quanti byte ha letto. Arrivati alla fine si riparte dall'inizio. Il lettore scrive senza passare dalla cache e, prima di spostare il puntatore, fa svuotare il buffer della cache L2 (PL310): l'FPGA legge la DDR e non vede le cache dell'ARM.

## Le caselle della posta di mc_acc

Il lettore scrive le sue parole: la firma `MCAC`, il numero di sessione, dove sta l'anello e fin dove ha scritto. Il core risponde da +64 byte con la firma `ACOK`, i byte letti, le immagini finite, i tempi e la parola 20, che dice che cosa sa fare:

| Bit | Significato | Bit | Significato |
|---|---|---|---|
| 0 | comando sbagliato al motore | 18 | filtro anti-blocchi nel core |
| 1 | la memoria non risponde | 19 | il core mescola menu e video |
| 2 | slice CABAC con errori | 20 | il core ridimensiona (720p...) |
| 16 | core 3.0 o più recente | 21 | acceleratore a 100 MHz |
| | | **22** | **acc_parse presente (A6)** |

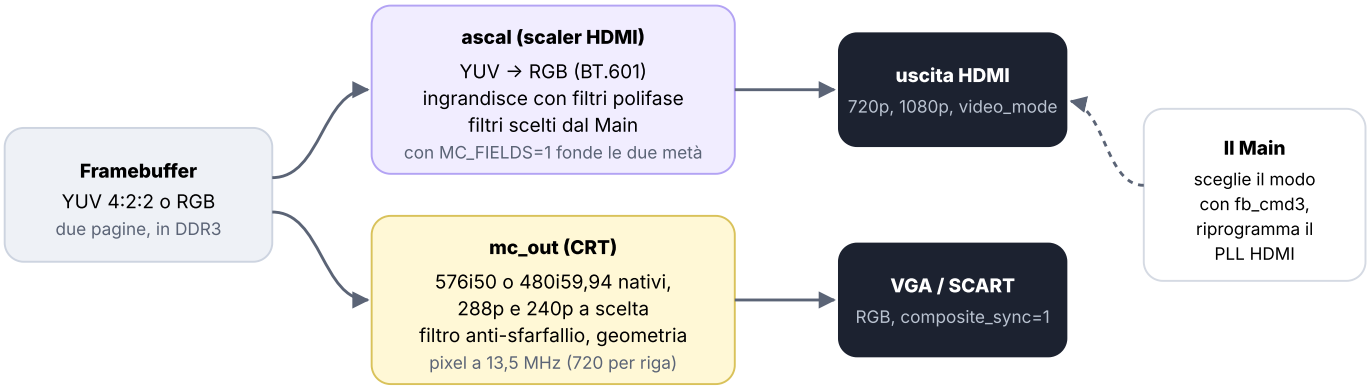
## Il comando A6

| Parole da 32 bit | Contenuto |
|---|---|
| 1 | A6 e il numero di parole che seguono |
| 5 | descrittore: slot dell'immagine e della co-locata, dimensioni, primo macroblocco, tipo di slice, QP, liste, filtro, offset |
| 0 o 96 | solo B con diretta temporale: distanze e mappa dei riferimenti |
| variabile | solo con pesi impliciti: i pesi per ogni coppia di riferimenti |
| 24 (+32) | matrici di quantizzazione 4×4 (e 8×8) |
| 1 + n | numero di byte, poi i byte della slice dopo l'allineamento CABAC, 4 per parola |

Tutti i comandi del motore (A0 riempimento, A5 filtro, macroblocchi...) sono descritti in `accel/doc/comandi.md`.

# 11 Le uscite HDMI e CRT, i registri

Lo stesso framebuffer va a due televisori insieme: l'HDMI attraverso lo scaler del MiSTer, il CRT attraverso l'uscita a 15 kHz scritta per MediaCenter.




## Impostazioni nel MiSTer.ini

```
[MiSTer]
vga_scaler=0
forced_scandoubler=0
composite_sync=1      ; con la SCART RGB

[menu]
main=MiSTer_MediaCenter
PHYSICAL_DISC_BOOT=1

[A0CD-*]
main=MiSTer_Physical-CD  ; giochi su disco
```

Il file `MMC_MiSTer_ini.txt` nello zip contiene queste righe da copiare. Per i giochi su disco serve il Main di MiSTer Physical Disc (Anime0t4ku), da scaricare a parte.

## Impostazioni utili in mediacenter.ini

| | |
|---|---|
| `MC_FIELDS=1` | tiene i semiquadri dei video interlacciati (consigliato con il CRT) |
| `MC_CRT_MODE` | 0 auto, 1 576i, 2 288p, 3 480i, 4 240p |
| `MC_CRT_HSIZE…VPOS` | dimensioni e posizione sul CRT (anche da Regola CRT) |
| `MC_AV_OFFSET` | sposta l'audio rispetto al video |
| `MC_NOACC=1` | spegne l'acceleratore (tutto sull'ARM) |
| `MC_NOCABAC=1` | CABAC sull'ARM come nella 3.6 |
| `MC_JELLYFIN=` | indirizzo del server Jellyfin (vuoto: lo cerca; 0: nasconde la voce) |
| `MC_AUDIO_W=720` | schermata audio a tutta larghezza |
| `MC_DIAG=0` | niente righe di diagnosi nel log |

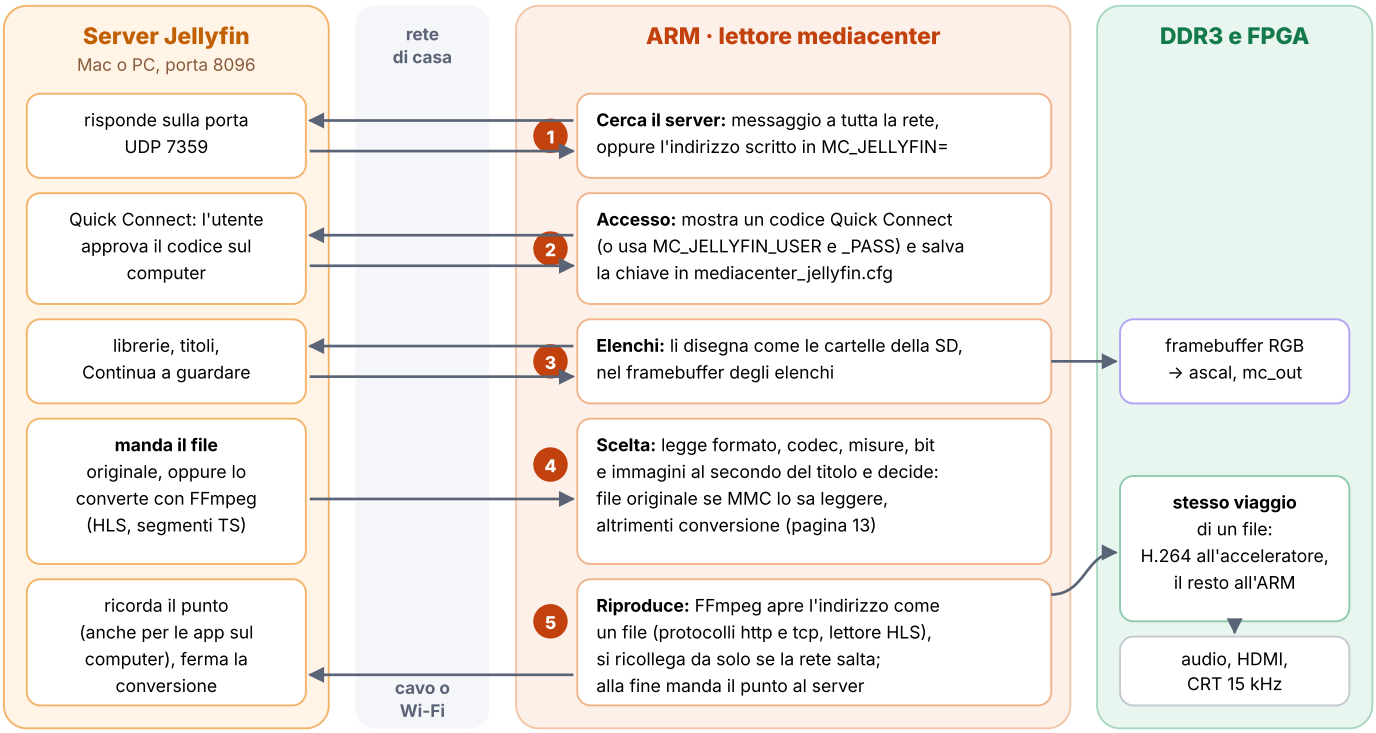
## I tre registri da leggere quando qualcosa non va

| File | Chi lo scrive | Cosa contiene |
|---|---|---|
| `/tmp/mediacenter.log` | il lettore (si riscrive a ogni video) | formato del video, strada scelta (*acceleratore acceso, CABAC nell'FPGA*; con Jellyfin *file originale* o *convertito dal server*), ogni 10 s: immagini mostrate, non mostrate, in ritardo, perse dall'FPGA, KB per immagine, tempi dell'ARM e dell'FPGA, attesa della memoria |
| `/tmp/mediacenter_main.log` | il Main | quale core del menu è stato caricato e quale Main gestisce il menu |
| `/tmp/mediacenter_crt` | il Main | modo video del CRT, misure e posizione dell'immagine |

**Una riga buona, per confronto** (One Piece 1080p, 9/11): acceleratore acceso, CABAC nell'FPGA; 0,1% di immagini non mostrate, 0 in ritardo, 0 perse dall'FPGA, circa 15 KB per immagine, ARM 0,3 ms.

# 12 Il viaggio 5: Jellyfin, dalla rete al lettore

Jellyfin è un programma gratuito che gira su un computer di casa e tiene in ordine film, serie e musica. MMC gli parla con le richieste HTTP della sua interfaccia pubblica, le stesse delle app ufficiali, con un codice scritto apposta per MMC (`jellyfin.c`). Tutto avviene sull'ARM: per l'FPGA un film da Jellyfin è un video come gli altri.




## Le richieste, in ordine

| Passo | Richiesta a Jellyfin |
|---|---|
| 1 ricerca | "who is JellyfinServer?" in UDP sulla porta 7359, fino a 3 volte; il server risponde con indirizzo e nome |
| 2 accesso | `/System/Info/Public`, `/QuickConnect/Initiate`, `/QuickConnect/Connect`, `/Users/AuthenticateWithQuickConnect` (oppure `/Users/AuthenticateByName`) |
| 3 elenchi | `/UserViews` (librerie), `/UserItems/Resume` (Continua a guardare), `/Items?ParentId=` (fino a 2.000 voci) |
| 4 scelta | `/Items/{id}`: le *MediaSources* con i flussi video e audio |
| 5 flusso | `/Videos/{id}/stream?static=true` o `/Videos/{id}/master.m3u8`; musica `/Audio/{id}/stream` |
| resoconti | `/Sessions/Playing` all'inizio, `/Sessions/Playing/Stopped` alla fine, poi `DELETE /Videos/ActiveEncodings` se c'era una conversione |

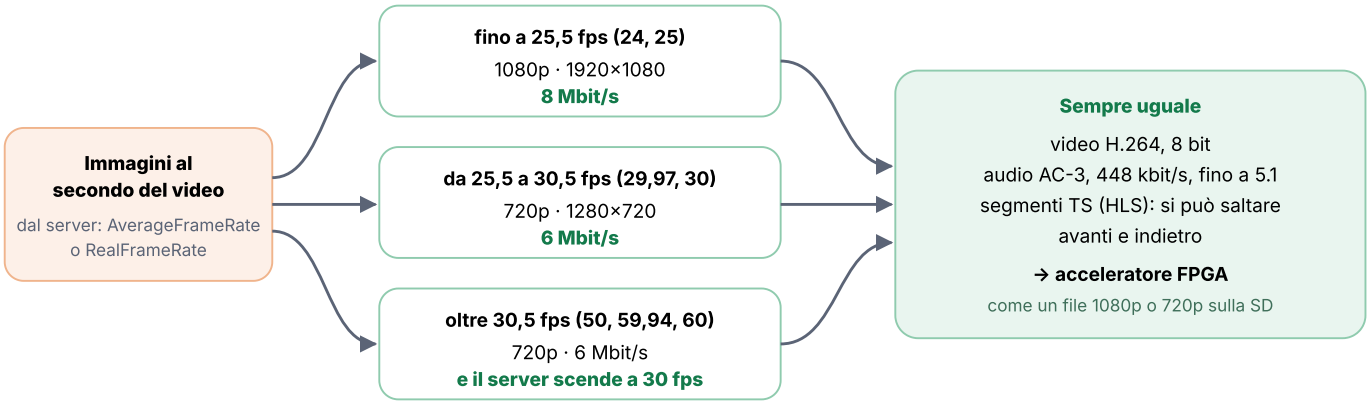
## Che cosa arriva originale

| | |
|---|---|
| **Video** | H.264 a 8 bit fino a 1920×1088, MPEG-1 e 2, MPEG-4 (DivX, Xvid), H.263, MJPEG, WMV, VC-1 |
| **Audio** | MP3, AC-3, E-AC-3, DTS, AAC, Vorbis, FLAC, ALAC, Opus, WMA, PCM |
| **Contenitori** | MKV, WebM, MP4, MOV, AVI, TS, MPEG, VOB, ASF, Ogg, WAV e i file audio |
| **Convertito** | HEVC, AV1, VP9, H.264 a 10 bit, oltre 1920×1088, dischi (immagini ISO, cartelle DVD o Blu-ray), audio come TrueHD |
| **Musica** | originale; quella che MMC non legge arriva come FLAC |

**Perché il file originale conta:** arriva lo stesso H.264 che starebbe sulla SD, quindi lavorano l'acceleratore FPGA, le tracce audio e i sottotitoli, il CRT e la schermata audio, senza perdere qualità.

# 13 Jellyfin: conversione, ripresa e numeri

Quando MMC non sa leggere un video (HEVC, AV1, 10 bit, 4K…), chiede al server di convertirlo in H.264 a 8 bit: così arriva di nuovo all'acceleratore FPGA. La richiesta è tarata su quanto lavoro l'acceleratore regge con margine.



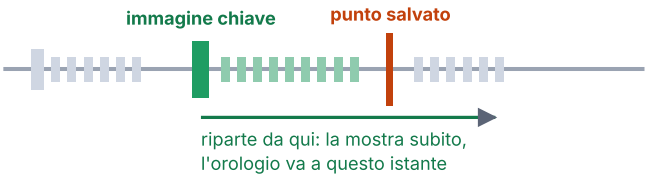

## Perché questi valori

Il tempo dell'acceleratore cresce con i **byte di ogni immagine**: più dati compressi vuol dire più lavoro per il lettore CABAC e più accessi alla memoria. Con la prima prova a 12 Mbit/s ogni immagine pesava circa 49 KB e l'FPGA impiegava 35–43 ms, più dei 33,4 ms che ha un video a 30 fps: il video scattava.

| Richiesta | KB per immagine | Tempo a disposizione |
|---|---|---|
| 12 Mbit/s a 30 fps (prima prova) | circa 50 | 33,4 ms: non basta |
| 8 Mbit/s a 24–25 fps | 40–42 | 40–41,7 ms |
| 6 Mbit/s a 30 fps | 25 | 33,4 ms |

KB per immagine = bit al secondo ÷ 8 ÷ immagini al secondo (valori medi; le immagini chiave pesano di più). Un video a 60 fps darebbe all'FPGA solo 16,7 ms per immagine: per questo il server lo porta a 30 ( `MaxFramerate=30` ).

## La ripresa dal punto lasciato




Prima: salto diretto al punto, 440 immagini scartate e buchi nell'audio.

Se Jellyfin ricorda un punto oltre i 30 secondi, MMC chiede *Riprendi da…* o *Dall'inizio*. Per riprendere, il lettore apre il video e riparte dall'immagine chiave che sta appena prima del punto: la decodifica e la mostra subito, mette l'orologio a quell'istante e riempie la riserva di immagini prima di partire. Così nessuna immagine arriva in ritardo. Il punto arriva da Jellyfin, quindi vale anche un film lasciato a metà sul computer.

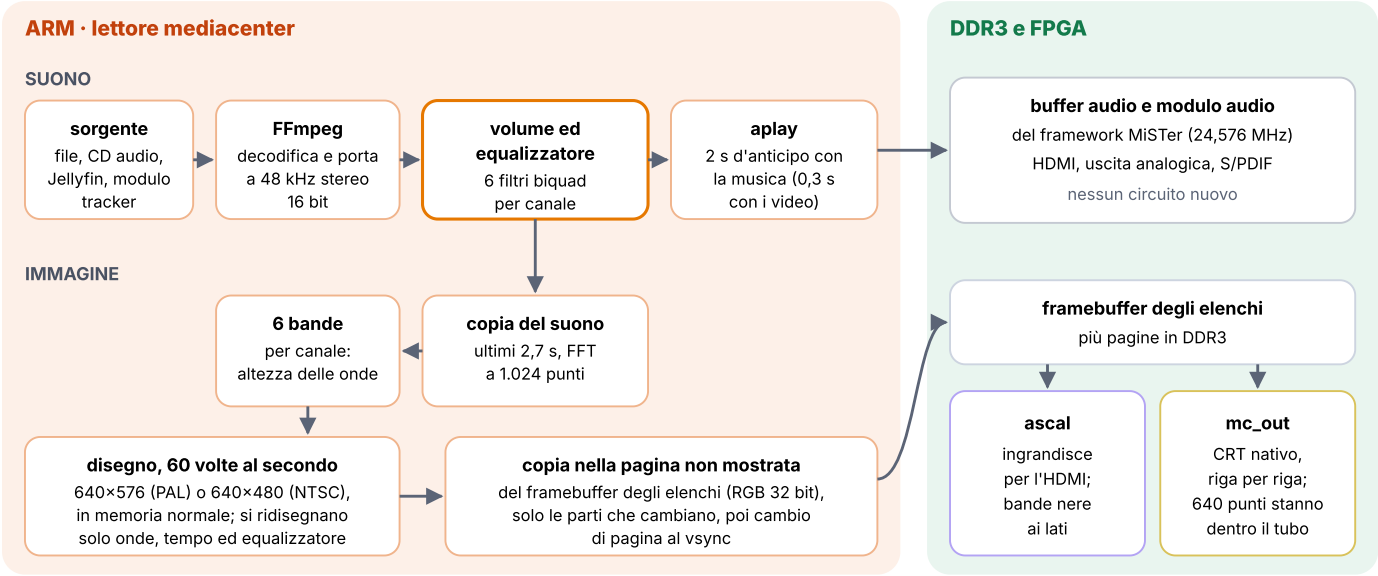
## Prove sul MiSTer, 11 e 12 novembre 2026 (server ufficiale su Mac mini M2 Pro)

| Video da Jellyfin | Strada | Risultato |
|---|---|---|
| Inuyasha, H.264 1440×810 MKV, 1.410 s | originale, acceleratore | 0,0% di immagini non mostrate, nessuna persa dall'FPGA |
| Tears of Steel, H.264 1728×720, 734 s | originale, acceleratore | 0,0% di immagini non mostrate |
| HEVC a 30 fps | convertito in 720p, 6 Mbit/s | 0,8% di immagini non mostrate |
| HEVC a 59,94 fps | convertito in 720p a 30 fps | 0,3% di immagini non mostrate |
| Ripresa dal punto lasciato | originale | 48 immagini pronte in 0,24 s, nessuna saltata, audio pulito |
| Musica FLAC e MP3 | originale | schermata audio, nessun problema |

**Quando la conversione pesa sul server.** Convertire un 4K HEVC è un lavoro grosso anche per un computer: nelle prove il Mac mini M2 Pro faticava sui 4K. Se il server non tiene il passo, il video scatta anche se il MiSTer è a posto. Per l'accelerazione hardware del server vedi la documentazione ufficiale di Jellyfin.

# 14 La schermata audio e l'equalizzatore

Quando suona un file audio, un CD o un brano da Jellyfin, l'ARM disegna una schermata viva: nome, cartella e formato, due onde che seguono la musica, il tempo, i comandi e un equalizzatore a 6 bande. L'FPGA la mostra come mostra gli elenchi.




## L'equalizzatore

| Banda | Filtro |
|---|---|
| 60 Hz | scaffale basso (alza o abbassa tutti i bassi) |
| 170, 470, 1,3k, 3,6k Hz | a campana (solo intorno a quella frequenza) |
| 10 kHz | scaffale alto (tutti gli acuti) |
| Vol | da 0 a −5 dB, per le canzoni registrate forti che sui televisori a tubo gracchiano |

Ogni banda va da −12 a +12 dB. Il calcolo costa all'ARM circa l'1–2% di un nucleo; con tutte le bande a 0 il suono resta identico. Le regolazioni restano per i brani seguenti e si salvano in `/media/fat/linux/mediacenter_eq.cfg`. L'equalizzatore lavora solo con la schermata audio, non sui video.

## I tasti

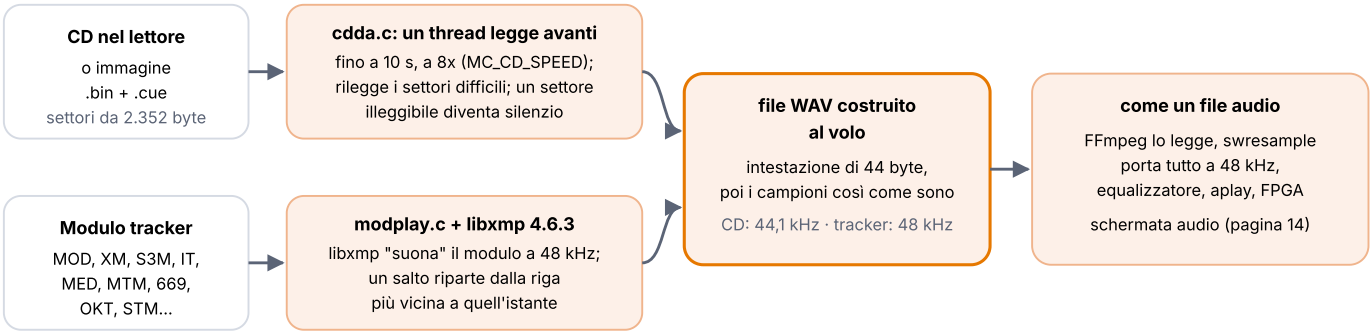
| | |
|---|---|
| **Y** | apre e chiude l'equalizzatore; si chiude da solo dopo 10 s senza tasti |
| **sinistra, destra** | con l'equalizzatore aperto: sceglie la banda o Vol |
| **su, giù** | alza o abbassa di 1 dB |
| **A** | salvaschermo acceso o spento (resta per i brani seguenti) |
| **B** | lascia il brano e ferma subito il suono: butta via quello mandato in anticipo e chiude aplay (lo stesso con brano seguente o precedente) |
| **pausa** | torna all'istante che si sente, così il suono si ferma subito |

**Impostazioni:** `MC_AUDIO_W=720` riporta la schermata a tutta larghezza; `MC_AUDIO_SIMPLE=1` rimette la schermata vecchia (solo titolo e formato). Con il CRT a 288p o 240p due righe del disegno diventano una (media).

**Perché 640 punti:** sul CRT la schermata a 720 punti finiva in parte fuori dal tubo; a 640 sta tutta dentro. HDMI e CRT leggono lo stesso framebuffer, quindi anche sull'HDMI compaiono due sottili bande nere ai lati.

# 15 CD audio, nuovi formati, Main, script e immagine

Il CD audio e i moduli tracker non sono file che FFmpeg sa aprire da solo. Il lettore li trasforma in un file WAV "finto", costruito al volo in memoria: così pausa, salti, avanti veloce, equalizzatore e schermata audio funzionano come per un MP3.




I file Opus non hanno bisogno di trucchi: FFmpeg ora contiene il decoder Opus e li apre come gli altri file audio.

## Il CD audio, dal disco allo schermo

1. Il Main vede un disco con sole tracce audio. Se in `MiSTer.ini` c'è `AUDIOCD=MEDIACENTER`, oppure non c'è nessun `AUDIOCD=` e MMC è installato, avvia il lettore con il nome del lettore CD.
2. Il lettore legge l'indice del disco e il **CD-Text** (titolo e artista di ogni brano); per un'immagine legge le righe TITLE e PERFORMER del `.cue`.
3. Tutte le tracce diventano un unico flusso, senza pause fra un brano e l'altro: le tracce sono i capitoli, sinistra e destra cambiano brano.
4. La schermata audio mostra numero, tempo e nome del brano.

Con `AUDIOCD=MISTERHIFI` il CD va a MiSTer Hi-Fi come prima.

## Il Main allineato a quello ufficiale

Il Main di MMC parte dal Main ufficiale del MiSTer con in più le parti di MMC. Nella 4.5 è stato unito al Main ufficiale dell'11 novembre 2026 (commit `04f91c6`, merge `0c6f8fe7`): arrivano la nuova mappatura dei controller **joymap v4**, il force feedback chiesto dai core e le novità di Minimig/CD32, Atari ST, Apple IIgs e degli altri core uscite dopo il 16 ottobre.

Restano come prima le parti di MMC: riconoscimento dei dischi, caricamento di `menu_mediacenter.rbf`, comando `fb_cmd3` per il formato dei colori, uscita CRT. Il core non cambia, quindi neanche il dialogo fra Main e FPGA.

## Lo script si chiama Mister_Media_Center.sh

Prima si chiamava `mediacenter.sh`: cambia solo il nome, il contenuto è lo stesso (MD5 `395f1811…` in entrambe le versioni). Il nome non ha spazi perché il Main avvia gli script con `cd $(dirname %s)` senza virgolette: uno spazio spezzerebbe il percorso. Chi aggiorna dalla 4.0 deve cancellare il vecchio `/media/fat/Scripts/mediacenter.sh`, altrimenti compaiono due voci.

## L'immagine di avvio

L'immagine *Ver. 4.5 H264 Jellyfin Client 1080 P* (`docs/mmc_splash.png`) è convertita da `make_splash.py` in `splash.h`: 640×480 punti, 256 colori, compressa a ripetizioni (RLE). Il lettore la disegna nel framebuffer degli elenchi per 3 secondi, con il numero di build in basso a destra, **solo quando parte dagli Scripts**. Un disco avviato dal Main va subito al film o al CD. `MC_SPLASH=0` la spegne; un tasto la salta.

# 16 Ricostruire tutto da zero (1/2): ingredienti e core

MiSTer MediaCenter è fatto di tre programmi e un circuito. Tutti nascono dallo stesso repository, alla release v4.5. Con questa pagina e la successiva si rifà ogni file dello zip.

| File dello zip | Cartella sulla SD | Da dove nasce | MD5 della release 4.5 |
|---|---|---|---|
| `menu_mediacenter.rbf` | /media/fat/linux/ | Quartus, cartella `fpga/menu` | `7cb8175f995d5b72a49c2ef032fbc963` |
| `mediacenter` | /media/fat/linux/ | `make mediacenter` | `c46efb131ba5947eaced0a9c08c89de6` |
| `MiSTer_MediaCenter` | /media/fat/ | `make` (il Main) | `eb6e97a3a906a8ce4d172bb913a27d69` |
| `Mister_Media_Center.sh` | /media/fat/Scripts/ | `Scripts/` del repository | `395f18116ad0a3156ccc3500e551e2d2` |
| `libdvdcss.so.2` | /media/fat/dvdcss/ | `dvdcss/` del repository | `7a140afb27296b1a7417358bbcc44cfd` |
| `MMC_MiSTer_ini.txt` | /media/fat/ | `docs/` del repository | righe da copiare |

### Passo 0 · Il codice

```
git clone https://github.com/bonna79/MiSTer_MediaCenter.git
cd MiSTer_MediaCenter
git checkout 307a7c5      # la release v4.5: tutto quello che è nello zip
```

Il circuito del core è identico al commit `506375b`, quello provato sul MiSTer nella 4.0: da lì alla 4.5 le cartelle `fpga/` e `accel/` non sono cambiate. Sono cambiati i programmi dell'ARM, lo script, l'immagine di avvio e la documentazione.

### Passo 1 · Il core FPGA, identico bit per bit

Quartus è **deterministico**: con gli stessi sorgenti, la stessa versione, lo stesso seme e la stessa data scritta nel core produce lo stesso file. Se cambia una sola di queste cose cambia tutto il piazzamento, e con il chip pieno al 94% i tempi possono non chiudere più.

**Versione**
Quartus Prime **17.0.2** Build 602 Lite Edition (gratuita)

**Seme**
`set_global_assignment -name SEED 12` in `fpga/menu/menu.qsf`

**Data fissa**
`fpga/menu/build_date.txt` = `261027`, scritta nel core da `sys/build_id.tcl`

```
docker run --rm -v "$PWD:/build" -w /build/fpga/menu \
  theypsilon/quartus-lite-c5:17.0.2 \
  quartus_sh --flow compile menu.qpf
# immagine docker: sha256:42e6c06da3af486f9cfea74acb5c1af9b63a77d0f16e3bf093215216e209b7ad
md5sum fpga/menu/output_files/menu.rbf   # deve dare 7cb8175f995d5b72a49c2ef032fbc963
```

Il risultato identico è provato con questa immagine Docker (e su GitHub). Con un Quartus 17.0.2 Lite installato a mano apri `fpga/menu/menu.qpf`, compila e controlla l'MD5. In alternativa su GitHub: *Actions → Build MediaCenter menu core → Run workflow* (circa 35 minuti, file `.github/workflows/fpga.yml`). Rinomina `menu.rbf` in `menu_mediacenter.rbf`.

> **Se l'MD5 è diverso**, controlla nell'ordine: versione di Quartus, `SEED 12`, data `261027`, posizione del PLL `FRACTIONALPLL_X0_Y56_N0` in `menu.qsf`. Se si cambia anche una riga di Verilog, il seme 12 non vale più: vanno provati altri semi finché uno chiude tutti i tempi nei quattro modelli (lento e veloce, 100 °C e −40 °C) con `quartus_sta`.

Il file `fpga/menu/sys/emu_ports.vh` può comparire modificato in git per i fine riga (LF/CR+LF): Quartus lo legge uguale e il core non cambia. Il piazzamento completo, cella per cella, è nell'allegato `seme12_piazzamento.zip` della documentazione 4.0 (120.318 celle, report dei tempi, menu.qsf).

# 17 Ricostruire tutto da zero (2/2): lettore, Main, pacchetto

I programmi per l'ARM si compilano su un PC Linux (GitHub usa Ubuntu 22.04) con il compilatore incrociato di ARM. Con gli stessi strumenti il risultato è equivalente, anche se non sempre identico byte per byte.

### Passo 2 · Il compilatore

```
# strumenti di base (Ubuntu): git make patch curl wget bzip2 xz-utils zip pkg-config
# GNU Toolchain di ARM 10.2-2020.11, arm-none-linux-gnueabihf, per PC x86_64 (indirizzo esatto di build.yml)
wget -qO- "https://developer.arm.com/-/media/Files/downloads/gnu-a/10.2-2020.11/binrel/gcc-arm-10.2-2020.11-\
x86_64-arm-none-linux-gnueabihf.tar.xz?revision=d0b90559-3960-4e4b-9297-7ddbc3e52783&rev=d0b9055939604e4b\
92977ddbc3e52783&hash=0074C1529DE90C98726B80ED3EE0776C" | tar -xJ -C /opt
export PATH=/opt/gcc-arm-10.2-2020.11-x86_64-arm-none-linux-gnueabihf/bin:$PATH
```

### Passo 3 · Le librerie del lettore

```
# FFmpeg 6.1.2: lo script applica tools/mediacenter/ffmpeg-mediacenter.patch
git clone --depth 1 -b n6.1.2 https://github.com/FFmpeg/FFmpeg.git ffmpeg-src
tools/mediacenter/build_ffmpeg.sh ffmpeg-src "$PWD/ffmpeg-arm"
# libdvdread 6.1.3 e libdvdnav 6.1.1, per i DVD
curl -fsSL https://download.videolan.org/pub/videolan/libdvdread/6.1.3/libdvdread-6.1.3.tar.bz2 | tar -xj
curl -fsSL https://download.videolan.org/pub/videolan/libdvdnav/6.1.1/libdvdnav-6.1.1.tar.bz2 | tar -xj
tools/mediacenter/build_dvdnav.sh libdvdread-6.1.3 libdvdnav-6.1.1 "$PWD/dvdnav-arm"
# libxmp 4.6.3 (licenza MIT), per i moduli tracker
curl -fsSLO http://archive.ubuntu.com/ubuntu/pool/universe/libx/libxmp/libxmp_4.6.3.orig.tar.gz
# sha256 b189a2ff3f3eef0008512e0fb27c2cdc27480bc1066b82590a84d02548fab96d
tar -xzf libxmp_4.6.3.orig.tar.gz
tools/mediacenter/build_libxmp.sh libxmp-4.6.3 "$PWD/xmp-arm"
```

FFmpeg è compilato statico per Cortex-A9 con NEON e solo i decoder che servono: video MPEG-1/2/4, DivX, H.263, H.264, MJPEG, WMV, VC-1; audio MP1/2/3, AC-3, E-AC-3, DTS, AAC, Vorbis, FLAC, ALAC, **Opus**, WMA, PCM; sottotitoli DVD, SRT, ASS. Per Jellyfin ci sono la rete, i protocolli `http` e `tcp` e il lettore `hls`. La patch aggiunge a FFmpeg la modalità acceleratore: invece di ricostruire i pixel, scrive i comandi per l'FPGA (e il comando A6 del core 4.0).

### Passo 4 · Il lettore e il Main

```
make mediacenter FFMPEG="$PWD/ffmpeg-arm" DVDNAV="$PWD/dvdnav-arm" XMP="$PWD/xmp-arm" MC_BUILD=224
#   → bin/mediacenter   (MC_BUILD è il numero scritto sullo splash)
make
cp bin/MiSTer bin/MiSTer_MediaCenter
```

### Passo 5 · Il pacchetto e l'installazione

```
mkdir -p pkg/linux pkg/Scripts pkg/dvdcss
cp menu_mediacenter.rbf bin/mediacenter pkg/linux/
cp bin/MiSTer_MediaCenter docs/MMC_MiSTer_ini.txt pkg/
cp Scripts/Mister_Media_Center.sh pkg/Scripts/ ; cp dvdcss/libdvdcss.so.2 pkg/dvdcss/
cp LICENSE pkg/linux/mediacenter-LICENSE
(cd pkg && zip -r ../MiSTer_MediaCenter.zip .)
```

Estrai lo zip nella radice della SD, copia le righe di `MMC_MiSTer_ini.txt` nel `MiSTer.ini` e riavvia. Su GitHub tutto questo lo fa `.github/workflows/build.yml`: un commit su main con `[release]` nel messaggio pubblica la release, e prima controlla l'MD5 del core.

### Passo 6 · La prova finale

1. Il menu del MiSTer parte e in `mediacenter_main.log` c'è *MediaCenter menu core*.
2. Lo script mostra l'immagine con *Build 224*.
3. Un MP4 1080p: nel log *acceleratore acceso … CABAC nell'FPGA*, perse 0.
4. Il video lossless di prova `sega_intro_lossless.mp4` passa all'ARM e il logo si vede.
5. Un DVD originale: menu, tracce audio e sottotitoli.
6. Un Video CD e un DivX: fluidi, audio pulito.
7. Un CD audio e un film da Jellyfin.

# 18 Glossario

Le parole tecniche usate in questo documento, spiegate in modo semplice.

| Parola | Cosa vuol dire |
|---|---|
| **FPGA** | Chip programmabile: invece di un programma contiene un circuito che si può ridisegnare. Sul MiSTer è un Intel Cyclone V. |
| **Core (.rbf)** | Il disegno del circuito caricato nell'FPGA. |
| **ARM, HPS** | Il processore Cortex-A9 a due nuclei a 800 MHz dentro lo stesso chip; ci gira Linux. |
| **ALM, LAB** | Il mattoncino di logica dell'FPGA; 10 ALM fanno un LAB. |
| **M10K, DSP** | Memorie interne da 10 kbit; moltiplicatori veloci. |
| **Clock, MHz** | Il battito che fa avanzare il circuito: 100 MHz = 100 milioni di passi al secondo. |
| **PLL** | Generatore di clock: moltiplica un clock d'ingresso e lo divide. |
| **Seme** | Numero da cui Quartus parte per piazzare il circuito. Semi diversi danno piazzamenti e tempi diversi. |
| **Setup, hold** | I due controlli dei tempi: arrivare in tempo e non cambiare troppo presto. |
| **Ponte (FIFO)** | Coda che passa dati fra due zone con clock diversi. |
| **DDR3** | La memoria da 1 GB della scheda, condivisa da ARM e FPGA. |
| **Framebuffer** | La zona di memoria che contiene l'immagine da mostrare. |
| **H.264** | Il formato video della maggior parte degli MP4 e MKV. |
| **Macroblocco** | Quadrato di 16×16 pixel: un 1080p ne ha 8.160. |
| **CABAC, CAVLC** | I due modi in cui l'H.264 scrive i dati compressi; il CABAC comprime di più ed è più lento da leggere. |
| **Slice** | Un pezzo di immagine compresso a sé; spesso è l'immagine intera. |

| Parola | Cosa vuol dire |
|---|---|
| **I, P, B, IDR** | Immagini: da sola, da precedenti, da precedenti e successive; una I da cui si riparte da zero. |
| **IDCT** | Trasformata inversa: dai coefficienti alle differenze in pixel. |
| **Deblocking** | Filtro che nasconde i bordi fra i blocchi. |
| **YUV** | Luminosità più due componenti di colore; 4:2:2 e 4:2:0 dicono quanto colore si tiene. |
| **Slot** | Uno dei 64 spazi da 4 MB dove il core costruisce un'immagine. |
| **Zero-copy** | Il core copia l'immagine nel framebuffer da solo, senza l'ARM. |
| **MD5** | Impronta di un file: stessa impronta, stesso file. |
| **Jellyfin** | Programma gratuito che sul computer di casa tiene film, serie e musica e li manda in rete. |
| **Quick Connect** | Accesso a Jellyfin con un codice approvato sul computer, senza scrivere la password. |
| **HLS** | Video mandato a pezzi di pochi secondi con un indice ( `.m3u8` ): si usa per le conversioni. |
| **Mbit/s** | Milioni di bit al secondo: quanti dati compressi ha un video. |
| **Equalizzatore** | Alza o abbassa bassi, medi e acuti; qui 6 filtri *biquad* per canale. |
| **FFT** | Calcolo che divide un suono nelle sue frequenze: fa muovere le onde. |
| **CD-Text** | Nomi di album e brani scritti su alcuni CD audio. |
| **Modulo tracker** | Musica fatta di campioni e note (MOD, XM, S3M, IT). |
| **joymap v4** | La nuova mappatura dei controller del Main ufficiale. |

# 19 Mappa dei sorgenti e ringraziamenti

## Dove sta ogni cosa nel repository

| Cartella o file | Contenuto |
|---|---|
| `fpga/menu/menu.sv` | il corpo del core: PLL, collegamenti, menu |
| `fpga/menu/rtl/` | mc_acc, mc_zc, mc_out, mc_bus, mc_parse, mc_afifo, pll_acc, sdram, ddram |
| `fpga/menu/sys/` | framework MiSTer: ascal (con YUV), OSD, audio, hps_io |
| `accel/rtl/` | il motore: acc_engine, acc_mc, acc_idct, acc_dbk, acc_bs, acc_resid, acc_recon |
| `accel/parse/rtl/` | acc_parse, cab_dec, tabelle `.svh` e `.hex` |
| `accel/sim/`, `accel/parse/sim/` | simulazioni e confronti con FFmpeg |
| `accel/doc/comandi.md` | tutti i comandi del motore |
| `tools/mediacenter/` | il lettore: mediacenter.c, dvd.c, accel.c, accel_link.c, zerocopy.c, **jellyfin.c, audio_ui.c, cdda.c, modplay.c**, splash.h, patch di FFmpeg, script delle librerie |
| `tools/vcd_player/` | il vecchio lettore Video CD, incluso nel lettore ( `MC_VCD_OLD=1` ) |
| `*.cpp` nella radice | il Main: user_io.cpp carica il core, video.cpp gestisce il framebuffer e il CRT |
| `support/physical_disc/` | riconoscimento dei dischi, CD audio verso MMC |
| `Scripts/` | `Mister_Media_Center.sh` |
| `docs/` | schede tecniche, immagini, righe per il MiSTer.ini |
| `.github/workflows/` | `build.yml` (programmi e release), `fpga.yml` (core) |

**Grazie** a Umberto Parisi (rmonic79) dei Beceri Videoludici, il cui lavoro su MiSTer CRT Adjust ha reso possibile Regola CRT; a Max Riptide per le tante prove; a mad4gith per le idee; a FDA per il supporto tecnico e l'aiuto a chiudere il seme 12. MiSTer Physical Disc è di Anime0t4ku. Il framework e il Main vengono da MiSTer di Sorgelig e dei suoi collaboratori.

**Stato al 12 novembre 2026.** Versione 4.5 provata sul MiSTer con CRT Sony Trinitron (SCART RGB) e HDMI insieme: controller con joymap v4, DVD originale, CD audio, DVD di dati, Opus, moduli tracker, schermata audio, Jellyfin con file originali (0,0% di immagini non mostrate), conversioni HEVC (0,3–0,8%), ripresa e musica. Le prove del core della 4.0 (pagina 9) restano valide: il core è lo stesso.

## Installare o aggiornare la 4.5

| File dello zip MiSTer_MediaCenter.zip | Cartella sulla SD |
|---|---|
| `MiSTer_MediaCenter` (Main) | `/media/fat/` |
| `mediacenter` (lettore ARM), `menu_mediacenter.rbf` (core) | `/media/fat/linux/` |
| `Mister_Media_Center.sh` (script) | `/media/fat/Scripts/` |
| `libdvdcss.so.2` | `/media/fat/dvdcss/` |
| `MMC_MiSTer_ini.txt` (righe da copiare nel `MiSTer.ini`, pagina 11) | `/media/fat/` |

**Dalla 4.0:** estrai lo zip sopra i file vecchi, cancella `/media/fat/Scripts/mediacenter.sh` e riavvia. `MiSTer.ini` e `mediacenter.ini` restano come sono. Il server Jellyfin si installa seguendo la documentazione ufficiale (jellyfin.org/docs).

**In caso di problemi** apri una segnalazione su github.com/bonna79/MiSTer_MediaCenter/issues con una foto dello schermo e i file `/tmp/mediacenter.log`, `/tmp/mediacenter_main.log` e `/tmp/mediacenter_crt`.